

Smart Student Budget & Expense Advisor

project report

Programming in java

CSE2006

Fall semester (2026 – 2027)

Submitted by :

Name : Asmi Ranaut

Registration No. : 25BAI11203

Branch: CSE(AI & ML)



**SCHOOL OF COMPUTING SCIENCE ENGINEERING
AND ARTIFICIAL INTELLIGENCE (SCAI)**

VIT BHOPAL UNIVERSITY

Introduction

Smart Student Budget & Expense Advisor is a simple full-stack web application designed to help students record their monthly budget and daily expenses. The system provides a dashboard showing the total amount spent and the remaining budget, along with simple rule-based budget advice.

The application was developed using Java and Spring Boot for the backend, MySQL for persistent storage, and HTML, CSS and JavaScript for the frontend. REST APIs connect the browser interface with the backend.

The project also demonstrates core Object-Oriented Programming concepts including encapsulation, abstraction, inheritance, polymorphism and interfaces. No AI or machine-learning component is used; the advisor is based on simple if/else rules.

Project Objective

- Provide a simple way for a student to store monthly budget information.
- Allow the student to add and view expenses.
- Calculate total spending and remaining budget.
- Provide simple budget advice based on spending.
- Demonstrate Java OOP and basic full-stack development concepts.

Problem Statement

Students often have a fixed monthly budget but may not maintain a clear record of how much they spend on food, travel, shopping, education and other activities. Without a simple record, it can be difficult to know the total amount already spent and the amount remaining.

The problem addressed by this project is therefore the lack of a simple, student-oriented application for recording expenses and comparing spending against a monthly budget.

Proposed solution: A lightweight web application where a student enters a monthly budget, records expenses, and receives an automatically calculated spending summary and simple budget advice.

Functional Requirements

ID	REQUIREMENT	DISCRIPTION
FR-01	Add Student	Store student name, email and monthly budge
FR-02	View student	Retrieve stored student records
FR-03	Update student	Modify student details and monthly budget
FR-04	Delete student	Delete student record
FR-05	Add expense	Store amount , category , description , date and student ID
FR-06	View expense	Display expenses to the student
FR-07	Delete expense	Remove an expense record
FR-08	Calculate spending	Calculate total expenses for the student
FR-09	Calculate remaining	Calculate monthly budget minus total spending

Non-functional Requirements

Requirement	Description
Usability	The interface should be simple enough for a student to enter and review expense
Performance	The application is designed for small-scale student expense records and normal local use
Maintainability	Backend code is separated into entities, repositories, services, controllers and advisor classes
Reliability	Basic exception handling is included for missing record
Security	Database credentials are kept in local application configuration and should not be committed to GitHub.
Portability	The application can run on a Java-supported development environment with MySQL installed
simplicity	The project avoids unnecessary frameworks and AI/ML components

System Architecture

The Smart Student Budget & Expense Advisor follows a simple layered architecture. The system consists of a frontend, backend, database, and budget advisor component.

Architecture Components

1. Frontend Layer

The frontend is developed using HTML, CSS, and JavaScript. It provides the user interface through which the student can enter student details, add expenses, view expense history, and see the budget summary.

2. Controller Layer

The controller layer is implemented using Spring Boot REST Controllers. It receives HTTP requests from the frontend and sends appropriate responses.

The project contains:

- StudentController
- ExpenseController
- AdvisorController

3. Service Layer

The service layer contains the main application operations and acts as a connection between controllers and repositories.

The project contains:

- StudentService
- ExpenseService

4. Repository Layer

The repository layer uses Spring Data JPA to communicate with the database. It performs operations such as saving, retrieving, updating, and deleting records.

The project contains:

- StudentRepository
- ExpenseRepository

5. Entity Layer

The entity layer represents the data stored in the database.

The project contains:

- Student
- Expense

6. Database Layer

The project uses MySQL to permanently store student and expense information. The database is named student_budget.

7. Budget Advisor Component

The advisor component provides simple rule-based advice based on the student's monthly budget and total spending. It uses an interface, abstract class, inheritance, and polymorphism.

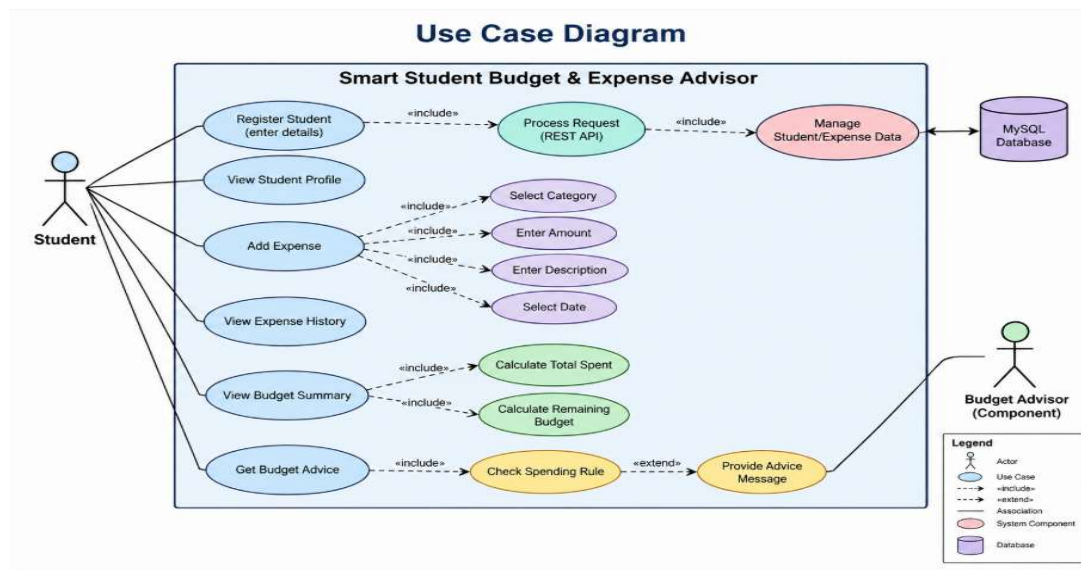
Architecture Flow

Browser → JavaScript → REST Controller → Service → Repository → MySQL

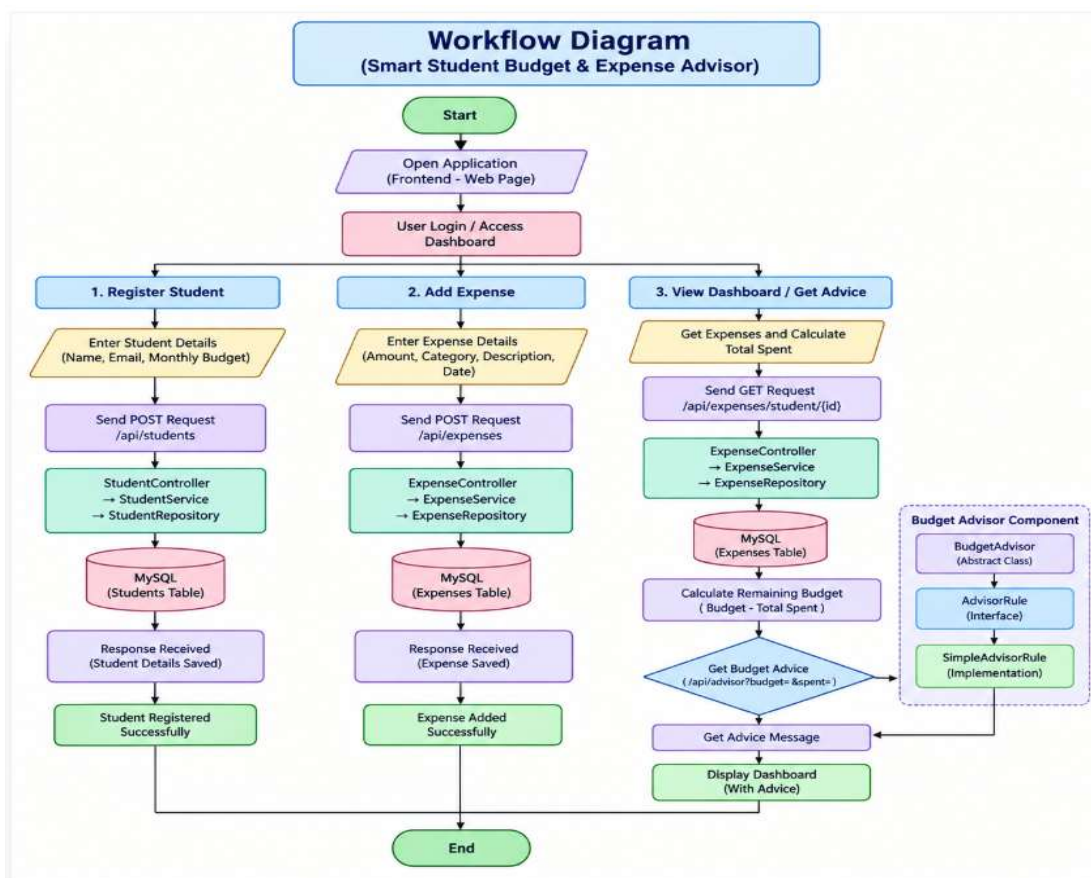
The advisor follow : Budget + Spending → Advisor Rule → Advice.

Design Diagrams

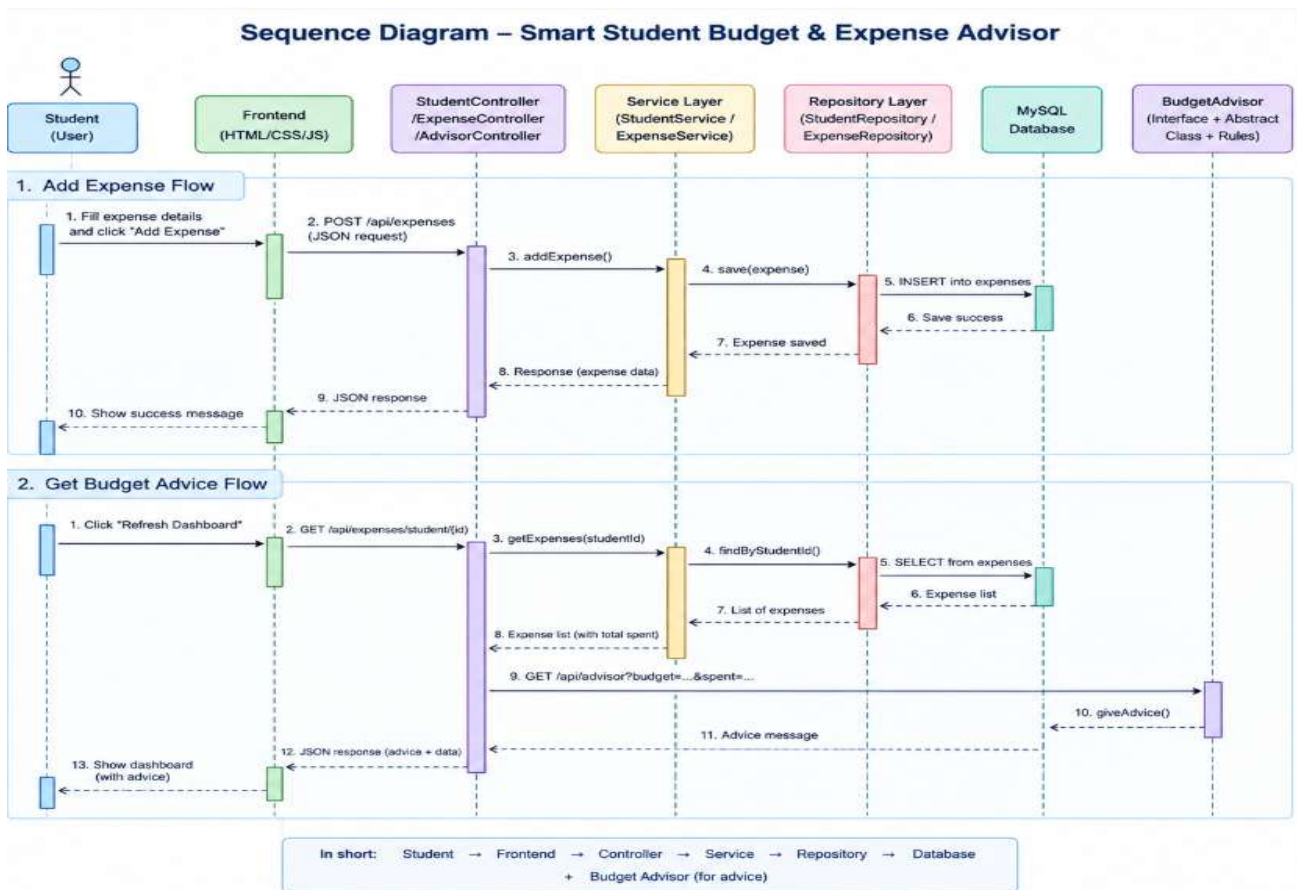
- Use case diagram



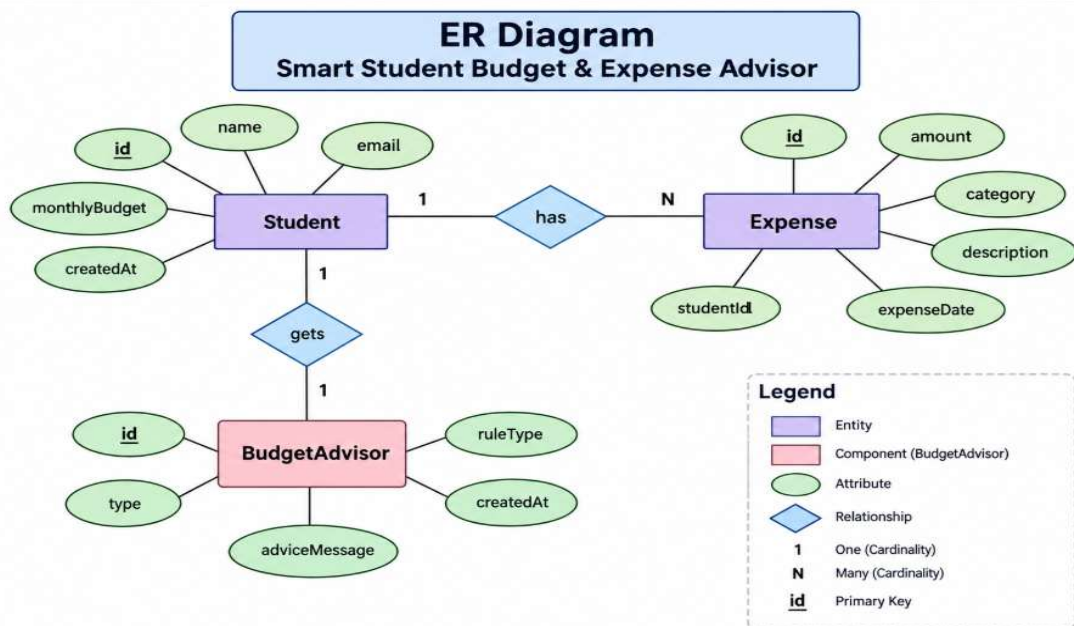
- Workflow diagram



- Sequence diagram



- ER Diagram



Design Decisions & Rationale

Decision	Rational
Java + Spring Boot	Provides a clear structure for a Java full-stack academic project and REST APIs
Spring data JPA	Reduces database boilerplate and provides repository methods such as findAll and findById
my SQL	Provides persistent relational storage for students and expenses.
Plain HTML/CSS/JavaScript	Keeps the frontend lightweight and easy to understand and demonstrat
Layered architecture	Separates controller, service, repository and entity responsibilities.
Student id in expense	Provides a simple association between expenses and students without making the JPA model unneces

Implementation Details

- **Student Entity**

The Student entity contains id, name, email and monthlyBudget. Private fields with getters and setters demonstrate encapsulation.

- **Expense Entity**

The Expense entity contains id, amount, category, description, expenseDate and studentId.

- **REST Controllers**

StudentController exposes CRUD endpoints under /api/students.

ExpenseController provides expense operations under /api/expenses.

AdvisorController exposes /api/advisor.

- **Services and Repositories**

Services contain application operations and repositories extend

JpaRepository. ExpenseRepository also

provides findByStudentId(Long studentId) for retrieving a student's expenses.

- **OOP Advisor Implementation**

AdvisorRule is an interface. BudgetAdvisor is an abstract class.

SafeBudgetAdvisor extends BudgetAdvisor, demonstrating inheritance.

SimpleAdvisorRule implements AdvisorRule. The statement

BudgetAdvisor advisor = new SafeBudgetAdvisor(rule); demonstrates polymorphism

- **Rule Logic**

Condition	Advice
spent > budget	You have exceeded your budget
spent >= 80% of budget	Warning: You have used most of your budget
otherwise	Good job! Your spending is under control

Screenshots / Results

Smart Student Budget & Expense Advisor

Student Details

Add Student

Add Expense

Add Expense

Budget Dashboard

Monthly Budget: ₹0

Total Spent: ₹0

Remaining Budget: ₹0

Refresh Dashboard

Advisor

Add your expenses to get advice.

Expense History

Amount

Category

Description

Date

Testing Approach

Testing was performed by running the Spring Boot application locally and sending requests through the VS Code REST Client. The frontend was then used to verify the integration.

Test case	Input/action	Expected result
TC-01	GET /api/students	Returns student list; initially [] if database is empty
TC-02	POST student with valid details	Student is saved and returned with generated ID
TC-03	POST expense for studentId 1	Expense is saved and returned with generated ID
TC-04	GET /api/expenses/student/1	Return expenses to student 1
TC-05	GET advisor with budget 10000, spent 5000	Return under control advice
TC-06	GET advisor with budget 10000, spent 8500	Returns warning advice
TC-07	GET advisor with budget 10000, spent 12000	Returns exceeded bedget advice
TC-08	GET missing student ID	Execption indicates student not found
TC-09	Frontend add student	Student is created and budget appears on dashboard
TC-10	Frontend add expense	Expense appears in history and total update

Challenges Faced

- Configuring the MySQL datasource correctly for Spring Boot.
- Creating the student_budget database and connecting it through application.properties.
- Understanding the difference between controller, service and repository responsibilities.
- Testing REST endpoints without Postman by using the VS Code REST Client
- Connecting the JavaScript frontend to the Spring Boot REST APIs.
- Implementing OOP concepts without making the project unnecessarily complex.
- Handling missing records through exception handling

Learnings & Key Takeaways

- Learned how to create a Spring Boot application and organize it into packages.
- Learned how REST APIs use HTTP methods such as GET, POST, PUT and DELETE.
- Learned how Spring Data JPA connects Java entities and repositories with a relational database.
- Practiced encapsulation, abstraction, inheritance, polymorphism and interfaces.
- Learned how frontend JavaScript can communicate with backend REST endpoints using fetch().
- Learned how to test APIs and verify database operations.
- Learned the importance of separating configuration and database credentials from public project documentation.

Future Enhancements

- Add charts for category-wise and monthly spending.
- Add authentication so multiple students can use separate accounts.
- Add monthly and category-based
- Add stronger validation for input fields.
- Improve responsive design for mobile devices.
- Add more configurable budgeting rules.
- Add automated unit and integration tests.

References

- Oracle Java Documentation — Java language and Object-Oriented Programming concepts.
- Spring Boot Documentation — Spring Boot application development and REST controllers.
- Spring Data JPA Documentation — repository and persistence concepts.
- MySQL Documentation — relational database and SQL concepts.
- MDN Web Docs — HTML, CSS, JavaScript and Fetch API concepts